

가능성을 제품으로 만들고, 끝까지 책임지는 Product Engineer 김대정입니다.

기획자로 시작해 백엔드로 왔고, 지금은 고객이 구독하는 AI 제품의 백엔드·AI 실행부를 직접 만들고, 핵심 화면은 coding agent로 완성해 운영합니다. AI 활용이 본격화되면서 모두가 메이커로 제품을 만들고 운영하는 팀이라, 백엔드 경험이 적은 팀원도 같은 기준으로 만들 수 있는 FastAPI 표준을 직접 세웠습니다.

피처링에서 먼저 확인할 네 가지

01 SNS 관측 데이터를 제품 기준과 평가 workflow로

게시물 데이터를 최신 상태·시계열·원문·사람 평가로 나누고, 다시 넣어도 결과가 같은 평가 workflow로 연결했습니다.

02 검증용 prototype을 실제 운영 backend로

돌아가는 prototype을 멈추지 않고 새 backend로 옮겨 실제 사용자 운영까지 이어갔습니다. 결함 재발이 약 94% 줄었습니다.

03 실패를 상태와 복구 흐름으로

외부 작업이 실패해도 주문이 멈추지 않도록 재시도·최종 실패·수동 재처리를 제품 흐름에 넣었습니다.

04 팀이 같은 기준으로 만드는 FastAPI 표준

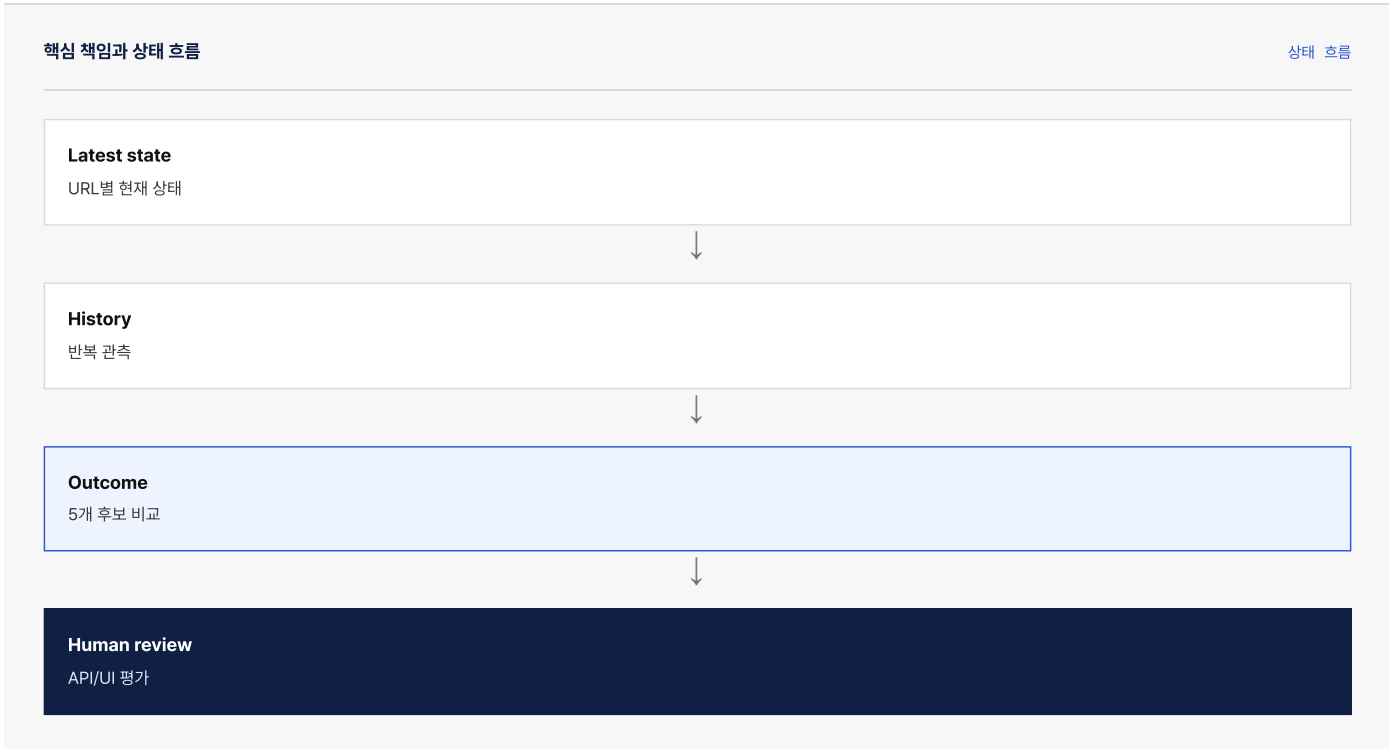
백엔드 경험이 적은 팀원도 같은 구조로 운영 제품을 만들 수 있게 transaction·session 기준을 template로 만들었습니다.

SNS 관측 데이터를 제품 기준과 재적재 가능한 평가 workflow로 바꿨습니다

직접 분석한 표본, 별도 시장 data의 latest-history, 원문 corpus와 사람 평가의 책임을 나눠 변경과 재처리가 가능한 workflow로 만들었습니다.

이 버전에서 보는 지점

직접 분석한 표본, 시장 data의 latest-history, local corpus 적재와 사람 평가를 구분해 봅니다.



문제 · 판단

하나의 숫자 대신 데이터의 생명주기를 나눴습니다

반응이 좋은 콘텐츠를 단일 threshold로 고정하지 않고, 최신 상태-시계열 관측-원문 corpus-사람 평가를 서로 다른 책임으로 분리했습니다.

Threads 게시물 4,039건을 직접 수집·분석해 생성 기준 수립

별도 시장 data의 latest 13.1만 행·observation 318만 행을 기반으로 절대·저자 상대·domain 상대·참여 품질·합의 기준의 5개 outcome 후보를 병렬 설계

운영 범위

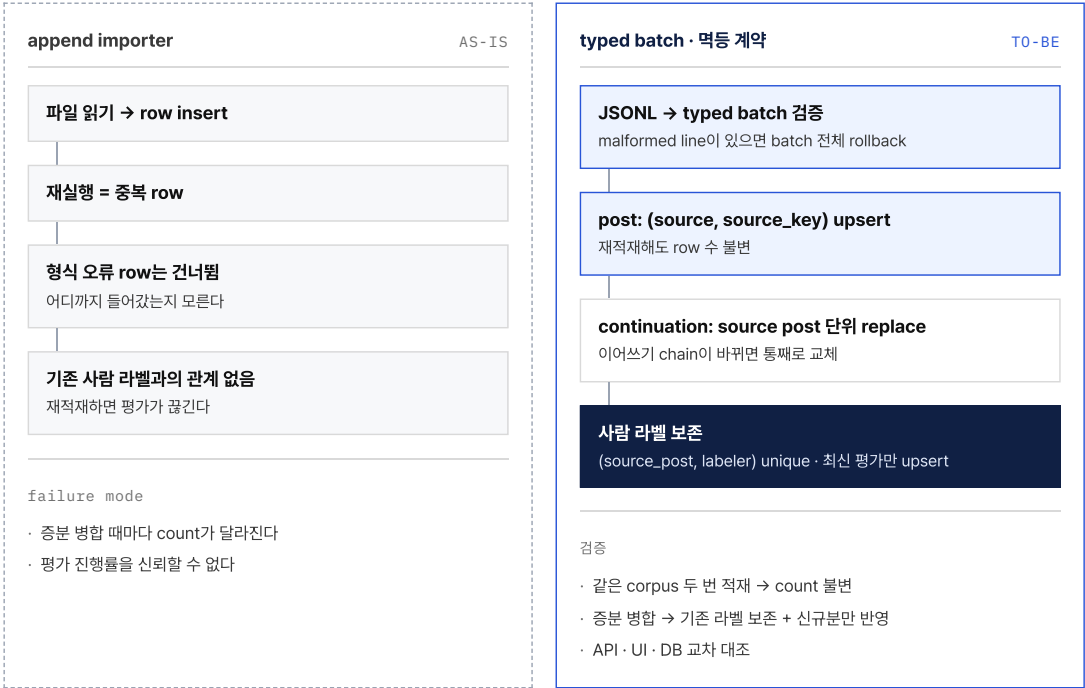
정제·적재·평가 구간을 재적재 가능한 workflow로 만들었습니다

수집기는 별도 파이프라인이고, 정제·적재·평가 구간을 설계·구현했습니다.

한국어 본문 11.1만 건·작성자 이어쓰기 18.5만 건을 독립 labeling schema로 이관

typed validation-source key upsert-continuation replace 기반 맥등 importer와 API/UI workbench 구축

malformed batch rollback-기존 label 보존-count 정합성을 local 전체 적재와 증분 재적재로 검증



불변 조건	아래층을 못 넘으면 위층을 묻지 않는다. 자동화는 바닥에만 돈다.
버린 대안	글자 수 상한 지시 (형태를 바꿔도 과교정을 유발 → 분포·총량 비례로 전환) · 자사 출력 값을 기준으로 삼는 것 (재실측으로 발견해 교정)
검증 근거	6축 점수 이력 · 소표본에서 대량 corpus로 측정 확대하며 유지·기각 결론 구분 · 판정 책임을 writer에서 planner로 옮긴 실측 (정보를 가진 자리로 책임 이동)
아래층을 통과하지 못하면 위층을 묻지 않는다. 자동화가 닿는 층과 닿지 않는 층을 처음부터 갈라 설계했다.	

담당 범위와 한계	피처링 JD와의 연결
수집기는 별도 파이프라인이고, outcome-schema-importer-API/UI workbench의 정제·적재·평가 구간을 직접 설계·구현했습니다.	피처링의 SNS data engine에 identity-history-idempotency-human evaluation을 분리한 경험으로 답할 수 있습니다.

검증용 prototype backend를 실제 사용자가 쓰는 운영 구조로 전환했습니다

제품과 frontend를 유지하면서 부분 수정과 backend 병렬 재구축을 비교하고, 검증 harness와 cutover 기준으로 팀의 합의를 만든 뒤 FastAPI backend를 교체했습니다.

이 버전에서 보는 지점

부분 수정과 병렬 재구축의 비교, 팀 설득, validation harness, cutover와 실제 사용자 운영을 봅니다.

핵심 책임과 상태 흐름

상태 흐름

Prototype

빠른 기능 검증



Inventory

API-기능 비교



v1.1.0

Backend cutover



v1.3.0

실제 사용자 운영

대안 · 선택

부분 수정과 병렬 재구축의 비용을 먼저 비교했습니다

기능 검증에는 빨랐던 초기 backend의 domain 의존성과 회귀 위험이 커졌습니다. 서비스가 작고 AI module 확장이 예정된 시점이어서 backend 범위만 병렬 재구축했습니다.

API-기능 inventory와 응답 비교 harness를 팀의 합의 근거로 사용

FastAPI backend 경계를 새로 두고 기존 frontend는 유지

coding agent는 codebase 분석·inventory·반복 구현에 활용하고 architecture·검증·cutover 판단은 직접 소유

검증 · 결과

재구축 뒤 결함 재발이 약 94% 줄었습니다

기존 동작과 새 backend 응답을 비교하고 test acceptance를 통과한 범위부터 전환했습니다. 같은 기준의 Jira 집계에서 해결된 QA 이슈 재오픈 비율이 37%에서 11%로 낮아졌습니다.

패턴·계층·검증 하네스를 먼저 구성한 뒤 v1.1.0에서 backend cutover

재오픈 비율 37% → 11%, 재발 발생 일평균 약 94% 감소(4.5건 → 0.3건, Jira 236건 전수 집계)

v1.3.0부터 실제 사용자가 쓰는 backend를 계속 개발·운영

운영 전환 이후

제품 원장과 AI 실행 상태를 분리하고 전달 정합성을 검증했습니다

AI 실행부를 독립 FastAPI application-DB로 분리하고, 원장 변경과 같은 transaction에 기록하는 Transactional Outbox로 지연·중복·역순 전달이 최신 상태를 덮지 않게 했습니다.

lease 기반 claim-attempt token-delivery version fence·먹등 consumer-terminal failure 보존

STG 생성 이력 2,616건·품질 795건·추적 7,111건 이관을 MD5 fingerprint-FK orphan 0건으로 검증

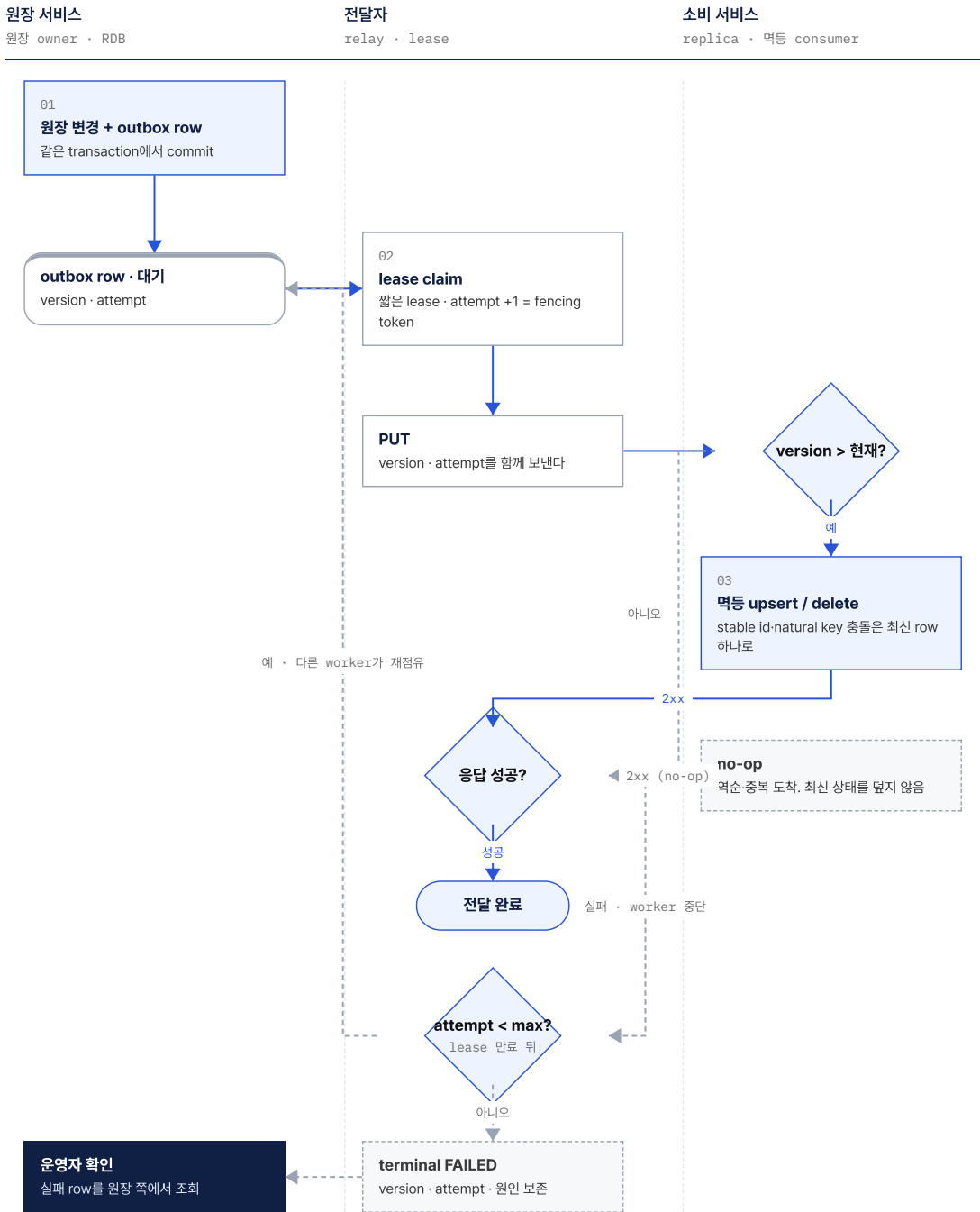
배포 성공과 기능 동작을 분리한 post-deploy 생성 API E2E gate

설계

prototype backend 재구축 — 돌아가는 기능을 왜 다시 만드는가에 대한 판단과 실행 순서

판단 비교 · 실행 순서





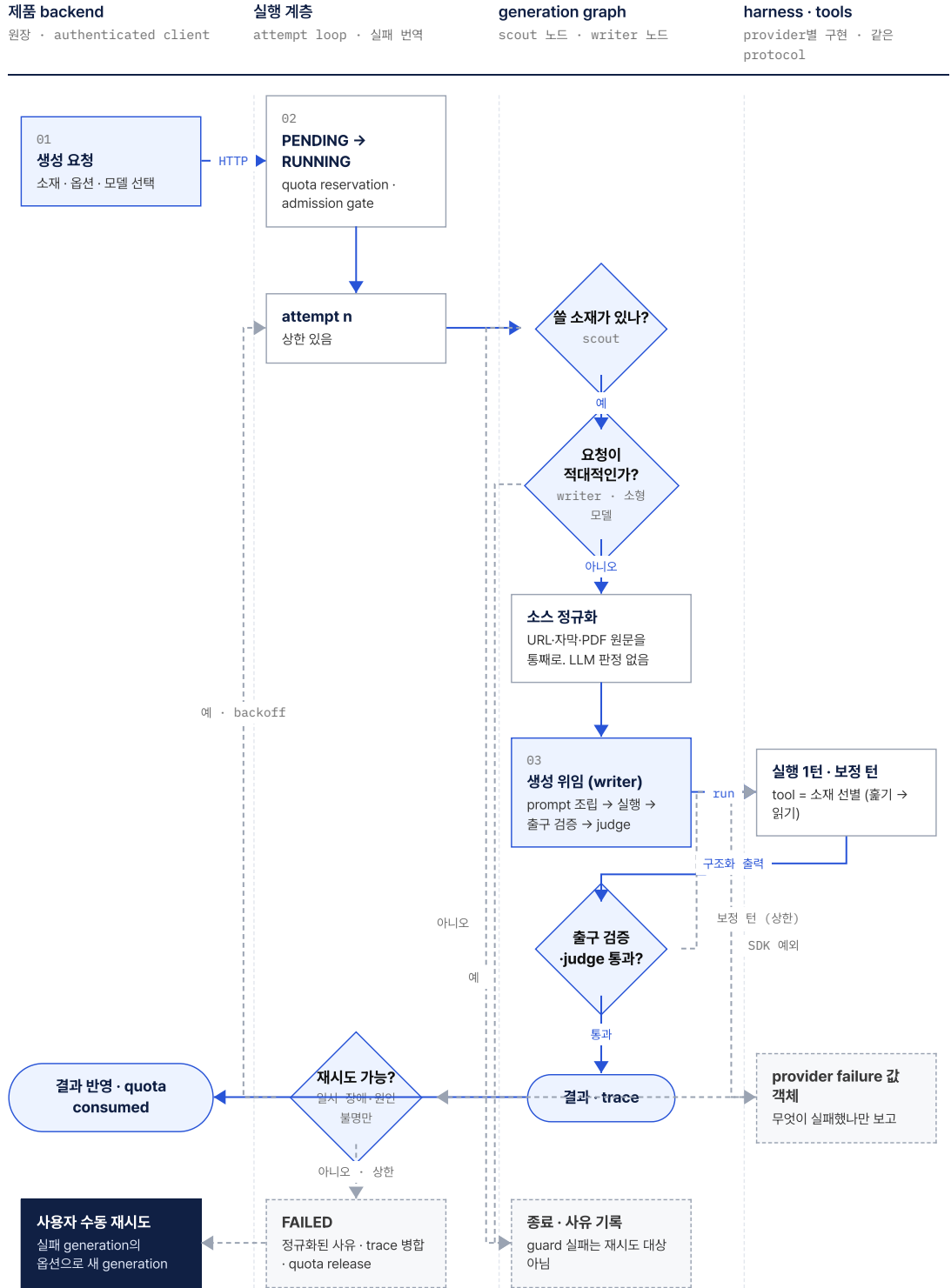
상태	진입 조건	이 상태에서 하는 일	성공 전이	실패 전이	관측·개입
대기 재시도 가능	원장 mutation과 같은 transaction에서 insert. 또는 attempt < max인 실패 뒤 lease 만료	아무것도 하지 않는다. relay가 claim할 때까지 row가 보존된다	relay claim → 점유	없음	대기 row 수 · 가장 오래된 row age
점유 lease 보유	relay가 짧은 lease로 row를 claim하고 attempt를 올린다	version·attempt를 fencing token으로 붙여 replica에 PUT/DELETE	consumer가 fence 통과 후 반영 또는 no-op → 전달 완료	응답 실패·worker 중단 → lease 만료. attempt < max면 대기로 복귀, = max면 terminal FAILED	lease 만료 뒤 다른 worker의 재점유 횟수
전달 완료	consumer 응답 성공	종료. 같은 version이 다시 와도 consumer가 no-op으로 끝낸다	없음 (terminal)	없음	version별 전달 지연
terminal FAILED	최대 시도 소진. 또는 중단된 마지막 claim의 lease 만료	version·attempt·원인을 보존하고 자동 재시도를 멈춘다	없음. 운전자 판단	없음	운영자가 원장 쪽에서 실패 row 조회

불변 조건	원장과 outbox row는 함께 commit되거나 함께 실패한다. replica는 더 높은 version만 받아들인다.
버린 대안	DB 공유(서비스 경계 붕괴) · dual-write(원장만 성공하고 전달 유실)
검증 근거	양쪽 서비스 전체 회귀, migration 왕복, stale PUT/DELETE fence test

열은 서비스 경계다. 실선은 정상, 점선은 실패 경로, 마름모는 판단, 짚은 박스는 사람 개입, 원통은 저장소.

AI 생성 파이프라인 — 생성 1건이 graph-역할 객체-harness를 지나 terminal 상태에 닿는 흐름

흐름 · 역할 객체 · 재시도



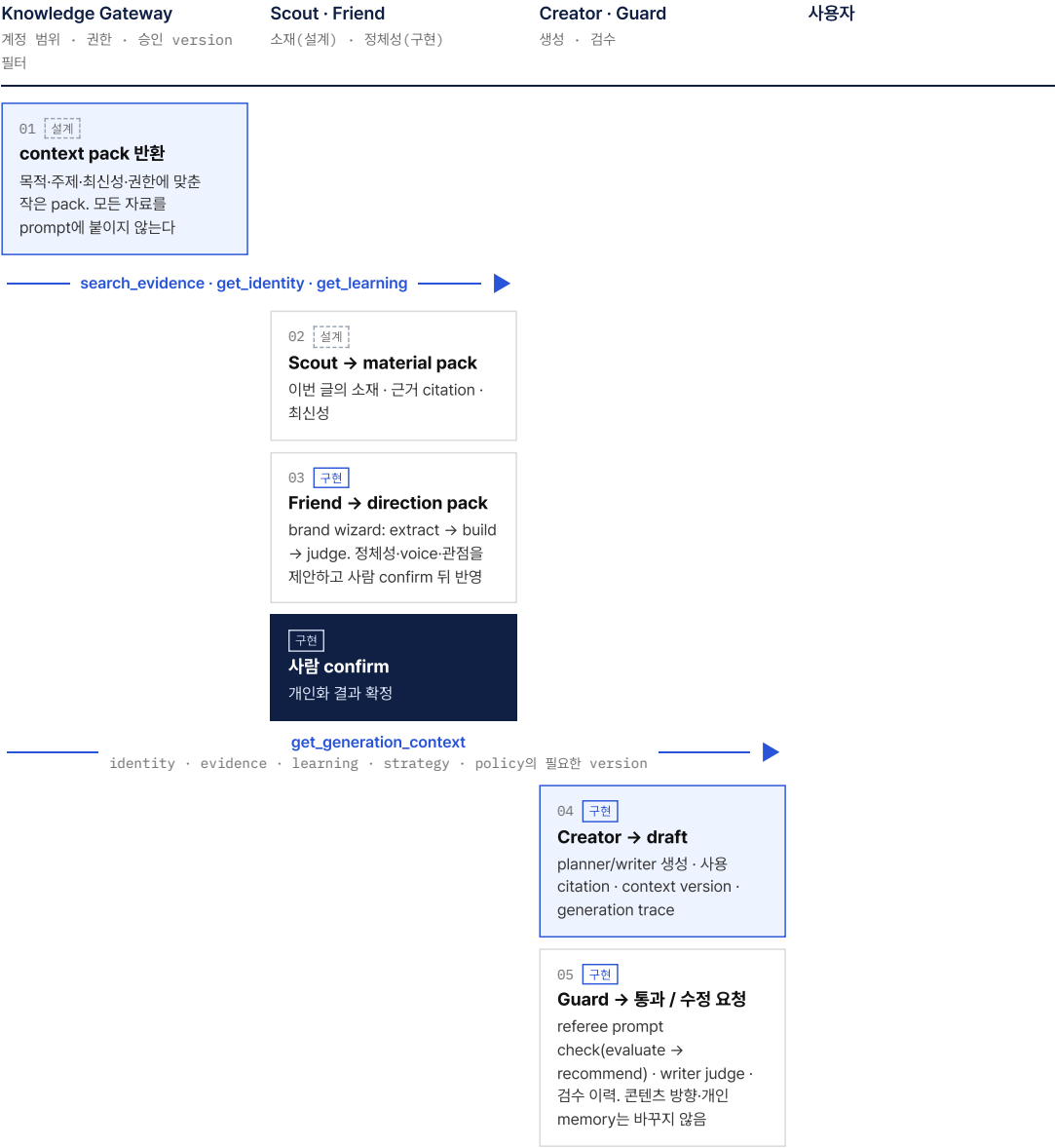
상태	진입 조건	이 상태에서 하는 일	성공 전이	실패 전이	관측·개입
PENDING	backend 요청 수락. quota reserved	worker 실행 대기. admission gate 계산에 포함	worker 시작 → RUNNING	취소 → CANCELLED	대기 수·복구 surface
RUNNING	worker 시작	attempt loop. graph 4노드·보정 루프·trace 기록	반환 가능한 결과 ≥ 1 → SUCCESS (quota consumed)	재시도 소진·비재시도 → FAILED / 사용자 취소 → CANCELLED	attempt·노드별 trace
SUCCESS	결과 생성	종료. 결과-citation-trace 보존	없음 (terminal)	없음	quota charge 대상
FAILED	결과 0개	정규화된 failure reason (source·provider·guard·output). quota release	사용자 수동 재시도 (새 generation)	없음	reason별 집계·오류 추적

불변 조건	역할은 LLM 지능만 소유한다. 판정은 그 입력을 아는 역할 안에 둔다 (소재 유무 = scout, 요청 적대성 = writer). 재시도 정책은 실행 계층이 소유한다.
버린 대안	plan → generate → fact guard 3노드 + repair loop의 구 graph 엔진 (agent 엔진이 전 환경 게이트를 통과한 뒤 폐기)·소스 map/reduce 압축과 LLM 유용성 분석·자동 circuit breaker-failover (당시 규모에서 운영 이점이 작다고 판단)
검증 근거	backend-AI application 전체 회귀·보정/judge 루프의 발행 계약 검증·5xx 분류·재시도-terminal failure 검증. friend-referee-scheduler 역할 객체도 같은 규칙으로 구현

역할은 LLM 지능만 소유하고 DB-transaction은 service가 소유한다. 재시도 여부는 AI 계층이 아니라 실행 계층이 정한다. provider 실명은 적지 않는다.

Thready 개인 계정 AX ☯ — agent는 gateway를 통해서만 승인된 지식을 읽는다

AX 설계 · 역할 그래프



학습 루프

설계

Threads 발행 → 반응

선택·수정·반응이 학습 데이터

judge · 성과

설계

learning change request

trace의 평가 결과. wiki를 직접
덮지 않는다

앞단 운영자 검토로 복귀

설계

승인분만 다음 projection

장기 memory 자동 승격 없음

역할별 계약 — 읽는 것 · 만드는 것 · 직접 바꾸지 않는 것

역할	읽는 정보 · tool	만드는 결과	다음 역할로	직접 바꾸지 않는 것	구현 상태
설계 Scout	search_evidence — 주제· 최신성·계정 범위의 승인 evidence	material pack (소재 · citation · 최신성)	Friend · Creator 입력	raw-extract 처리, identity-strategy-policy	자동 소재화 계약만 선언. 트리거·추가·승인 흐름 미확정
구현 Friend	get_identity · get_learning · search_evidence	direction pack (profile · voice · 관점 · 의도)	사람 confirm → Creator	raw 원문, 공통 strategy, 장기 memory 직접 승격	brand wizard graph(extract → build → judge)로 구현. gateway형 tool 계약은 설계
구현 Creator	get_generation_context — 필요한 version만 묶은 context pack	draft · 사용 citation · generation trace	Guard	wiki 장기 지식, policy, 발행 승인	writer 역할 객체 + repair/judge 루프로 구현
구현 Guard	draft · 적용 policy · evidence citation	통과 / 수정 요청과 이유	사용자 선택	콘텐츠 방향 · 개인 memory	결정적 출구 검증 + writer judge + referee prompt check(evaluate → recommend → finalize)

불변 조건	agent는 gateway가 돌려주는 승인 version만 읽는다. 평가 결과는 wiki를 직접 덮지 않고 앞단 검토로 되돌아간다. 발행 여부는 사용자가 정한다.
버린 대안	production Scouter를 먼저 만드는 것 (동의 범위·수집 주기·승인 흐름이 미확정) · 사람 확인 없는 자동 발행 · 장기 memory 자동 승격
검증 근거	설계안의 역할 계약 표 + 운영 코드의 friend(brand wizard) · writer · referee(prompt check) · scheduler(발행 시점) 역할 객체와 예약· 발행 화면. Scout의 production 구현은 주장하지 않는다

Scout는 설계 단계, Friend(brand wizard)-Creator(writer)-Guard(referee-judge)-발행 시점 제안(scheduler)-사용자 발행 선택은 운영 구현이 확인된 요소다.
사람 게이트는 세 곳: 운영자 승인, Friend 개인화 confirm, 사용자 발행 선택.

담당 범위와 한계

초기 prototype 이후 FastAPI backend 재구축·운영은 전담했고, 초기 prototype
구현은 다른 engineer의 범위입니다.

피처링 JD와의 연결

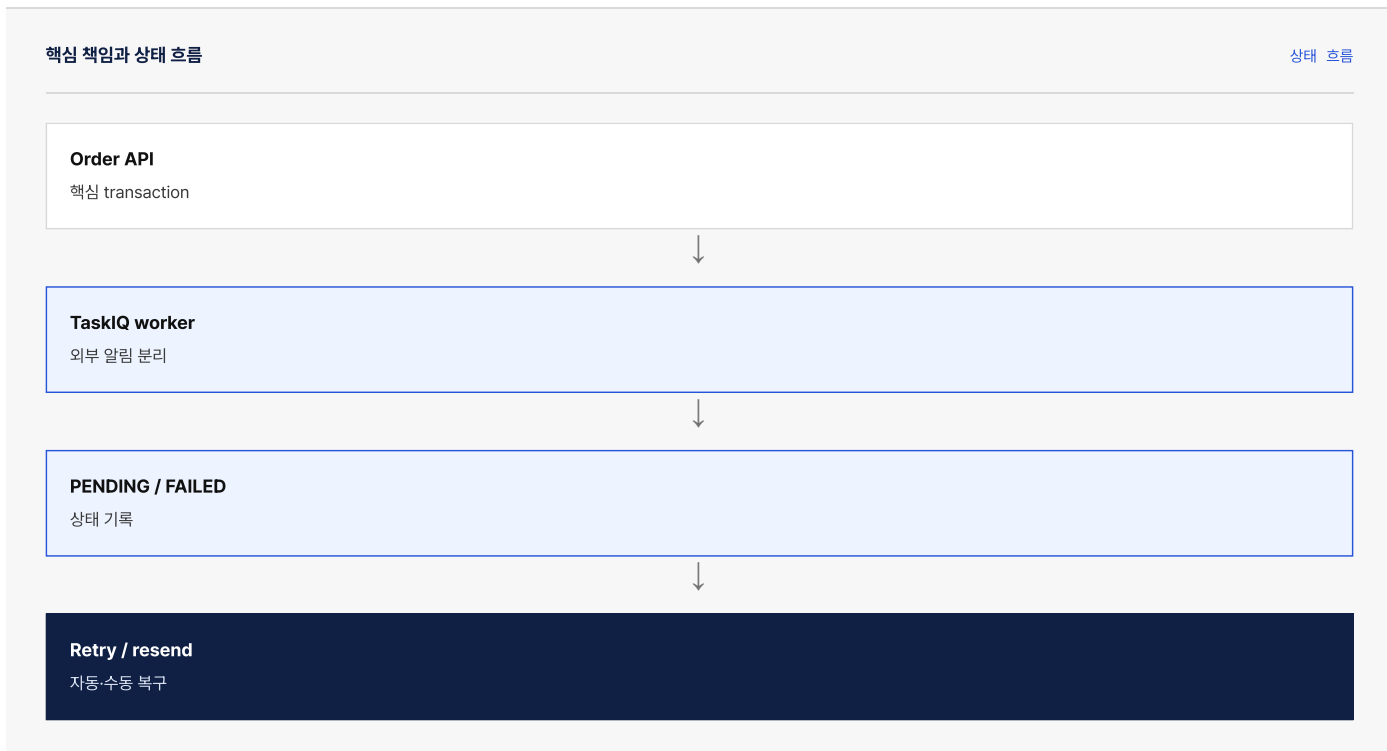
검증 중인 제품을 멈추지 않고 신규 backend로 전환하고 안정화한 판단·설득·
실행 경험입니다.

비동기 실패를 제품 상태와 재처리 흐름으로 드러냈습니다

Gateway·SSO·FastAPI service가 연결된 의료 MSA에서 주문·재고 API와 RabbitMQ·TaskIQ worker를 구축하고, 외부 알림 실패를 찾고 다시 처리할 수 있는 운영 경계로 만들었습니다.

이 버전에서 보는 지점

주문 transaction과 외부 작업을 분리하고 상태·retry·최종 실패·수동 재처리를 제품 흐름에 연결한 경험을 봅니다.



문제·선택

API 성공이 외부 작업 완료를 뜻하지 않게 했습니다

주문 transaction이 끝난 뒤 공급사·병원 알림은 별도로 실패할 수 있었습니다. API 요청 안에서 모든 작업을 끝내지 않고 worker로 분리했습니다.

async FastAPI 실행 모델과의 정합성을 기준으로 Celery에서 TaskIQ·RabbitMQ로 전환

알림 domain·service·worker와 worker image를 별도 배포 단위로 분리

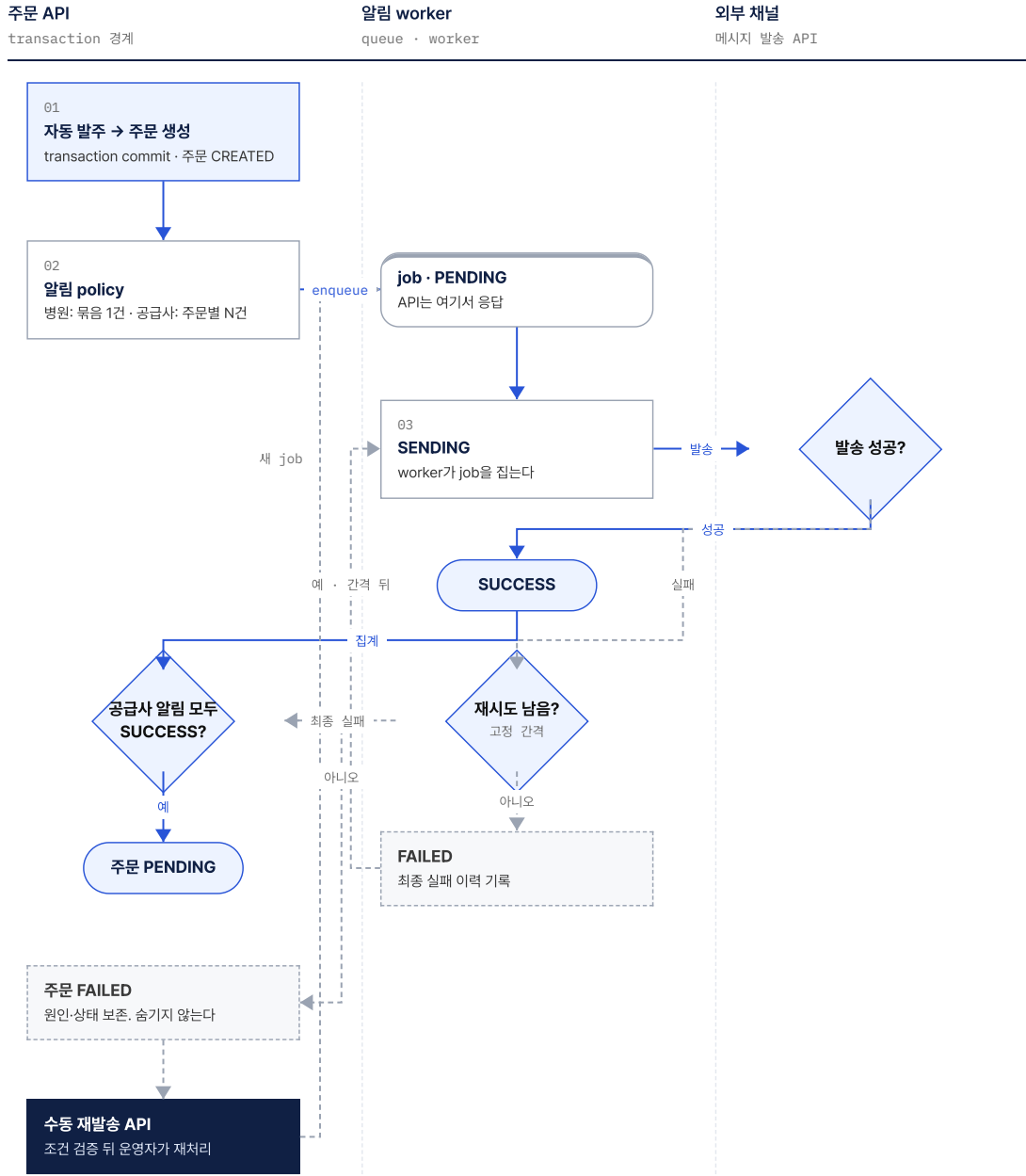
구현·검증

실패 상태와 복구 방법을 주문 흐름에 연결했습니다

발송 결과를 PENDING·FAILED 상태와 연결하고 자동 retry·최종 실패·수동 재발송 경계를 구현했습니다.

실패 기록을 남겨 API 성공 뒤 남은 작업을 다시 찾을 수 있게 구성

API test infrastructure·Docker CI와 local 실행·onboarding 문서 구축



알림 job · 주문 상태별 처리

상태	진입 조건	이 상태에서 하는 일	성공 전이	실패 전이	관측·개입
PENDINGjob	주문 transaction commit 뒤 알림 policy가 enqueue	queue에서 대기. API는 이미 응답을 돌려준 상태	worker 수신 → SENDING	없음	큐 적체
SENDINGjob	worker가 job을 잡는다	외부 발송 API 호출	응답 성공 → SUCCESS	오류 → 고정 간격 재시도. 상한 소진 → FAILED	시도 횟수 · 마지막 오류
FAILEDjob	재시도 상한 소진	최종 실패 이력 기록. 자동 재시도 없음	수동 재발송 → 새 job PENDING	없음	운영자가 실패 이력·재발송 조건 확인
CREATED주문	자동 발주 engine이 주문 commit	알림 policy 호출	공급사 알림 모두 SUCCESS → PENDING	하나라도 최종 FAILED → FAILED	CREATED에 머무는 주문 수
FAILED주문	공급사 알림 중 최종 실패 존재	원인·상태 보존	전용 API에서 조건 검증 뒤 수동 재발송	없음	운영자 화면에서 실패 주문 조회

불변 조건	API 응답은 알림 완료를 뜻하지 않는다. 실패는 숨기지 않고 주문 상태와 이력에 남긴다.
버린 대안	기존 task queue 유지(asyncio 실행 모델과 맞지 않음) · API 안에서 알림 완료를 기다리는 동기 처리
검증 근거	API test infrastructure · Docker CI · 재고 worker의 동일 retry 경계

실시간 전사 — 외부 모델 세션이 발화 중간에 끊겨도 전사가 멈추지 않는 구조

구조 · 시간축 · 실패 흡수



구성 요소별 책임 — 각 부품은 자기 실패만 안다

구성 요소	받는 것	책임	내보내는 것	실패했을 때	모르는 것
Slicer	연속 오디오	window가 stagger보다 길게 잘라, 어느 순간에도 두 세션 이상이 같은 구간을 듣게 한다	겹치는 window	없음 (결정적)	세션 상태
세션 풀	window	동시 N개 세션. 각 세션은 자기 window만 담당하고 끝나면 다음 window로 순환	세션별 전사 조각	세션 종료-timeout은 그 window 하나만 읽는다. 풀은 계속 돈다	다른 세션의 진행
세션별 버퍼	전사 조각	세션 안에서 순서를 지키고, 문장 종결 부호에서 COMPLETE로 자른다. 재연결 시 버퍼를 비운다	COMPLETE + sequence	부분 문장은 다음 조각과 이어 붙인다	다른 세션의 버퍼
병합기	여러 세션의 COMPLETE	세션이 아니라 sequence 기준으로 하나의 흐름으로 정렬한다	순서 있는 전사 스트림	빠진 sequence는 기다리지 않고 다음으로 넘긴다 (실시간 우선)	어느 세션이 죽었는지
하류 트리거	COMPLETE	advice-추천 판단을 시작한다	상답 조언	COMPLETE가 늦으면 늦게 시작할 뿐, 멈추지 않는다	세션 풀의 존재

불변 조건	세션 하나가 죽어도 전사는 멈추지 않는다. 겹침은 window > stagger에서 나오고, 순서는 세션이 아니라 sequence가 정한다.
버린 대안	단일 세션 + 재연결 (끊긴 구간이 그대로 유실되고, 재연결 타이머 경합이 약 1분 시점 응답 중단을 만들었다) · 모델 교체에 기다리며 기능을 세워 두는 것
검증 근거	개발 test로 풀 크기와 window-stagger를 정한 뒤 dev/stg에 적용. 동시 세션 수만큼 비용이 늘고 첫 응답이 늦어지는 손실은 문서로 명시. 모델 교체 뒤 레거시 provider와 함께 제거

외부 실시간 음성 모델의 세션이 통제할 수 없는 시점에 종료되던 문제를, 모델 교체가 오기 전까지 구조로 흡수했다. 빗금은 두 세션이 같은 구간을 듣는 겹침이고, 점선은 끊긴 세션이다. 모델 교체 뒤 이 우회 구조는 걷어냈다.

담당 범위와 한계

주문·재고 backend와 worker 구축은 주도했고, Gateway-SSO는 연동 범위에서 담당했습니다.

피처링 JD와의 연결

피처링 engine의 service 배포·운영과 데이터 작업 실패 복구를 상태·retry·재처리 관점에서 설명하는 근거입니다.

service·repository·transaction 책임을 실행 가능한 기본값으로 만들었습니다

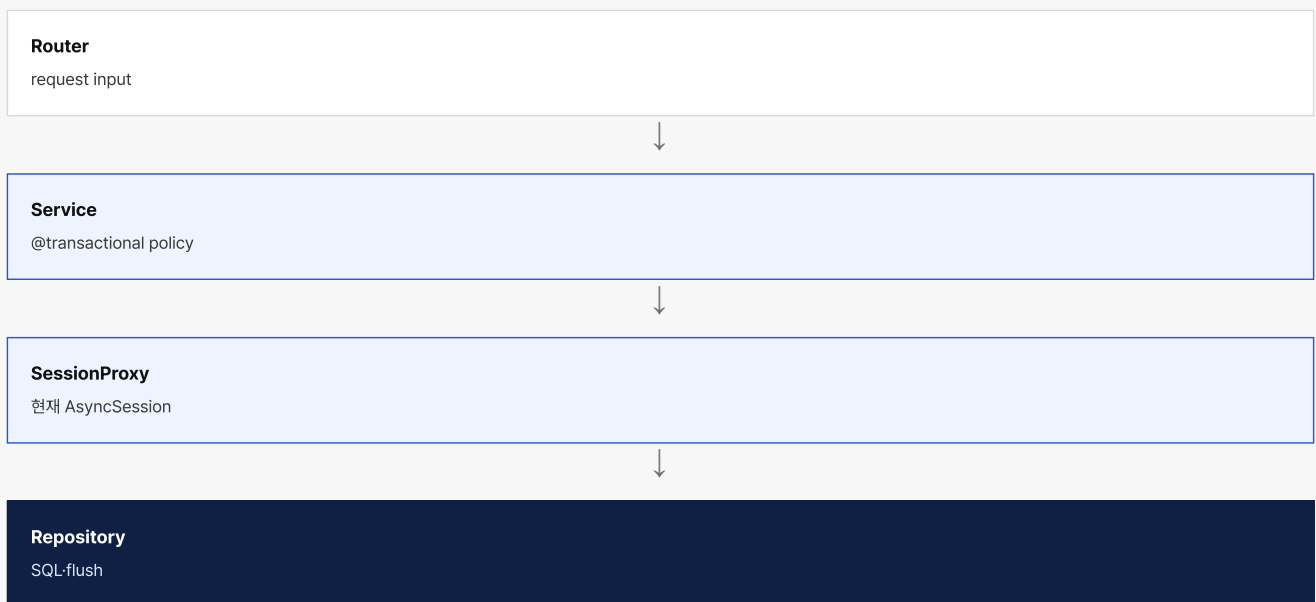
여러 FastAPI service가 entity·business logic·DB 접근과 transaction 책임을 다르게 해석하지 않도록 조직 template에 계층·DI·session ownership·검증 기준을 구현했습니다.

이 버전에서 보는 지점

entity·service·repository·transaction 책임과 AsyncSession ownership을 반복 가능한 조직 기준으로 만든 경험을 봅니다.

핵심 책임과 상태 흐름

상태 흐름



문제 · 판단

session 전달 반복보다 transaction 책임의 위치를 먼저 정했습니다

Router에서 받은 session이 모든 계층의 signature를 통과하고, transaction을 참여·분리·중첩할 위치가 호출부마다 달라질 수 있었습니다.

Router·Service·Validator·Repository·Model과 DI로 책임과 의존 방향을 고정

Repository는 SQL 실행·flush, Service는 transaction policy를 소유

failure mode · 검증

편의보다 session ownership과 실패 복구를 예측할 수 있게 했습니다

ContextVar·SessionProxy로 현재 AsyncSession을 resolve하되 하나의 transaction은 하나의 asyncio task와 session이 소유하도록 제한했습니다.

REQUIRED-REQUIRES_NEW-NESTED propagation과 SAVEPOINT 동작 구현

child task의 동일 session 접근을 fail-fast

cancellation rollback-connection cleanup-transaction propagation을 integration test로 검증

병렬 DB 작업의 데이터 가시성-복구-connection 비용을 ADR-runbook에 기록

설계



담당 범위와 한계 FastAPI-SQLAlchemy 조직 template은 직접 구축·적용했고, 모든 service의 동일 version 전환을 의미하지는 않습니다.	피처링 JD와의 연결 피처링 JD의 entity 구조-modular development-test 요구를 구체적인 transaction/session contract와 검증 방식으로 보여줍니다.
--	--